Taras Shevchenko National University of Kyiv
Faculty of Computer Science and Cybernetics
Master, 2nd year

LABORATORY WORKS REPORT

Genetic Algorithms and Discrete Search

Course: Mathematical Methods of Artificial Intelligence

Performed by: Matvii Kovalyk

Kyiv, 2026

Contents

Introduction 3
1. Maximizing a Function of Four Variables 4
1.1. Problem statement 4
1.2. Conceptual solution 4
1.3. Problems encountered and how they were fixed 5
1.4. Interesting observations 6
1.5. Results of execution 8
1.6. Test cases 8
1.7. Conclusion 9
2. The Travelling Salesman Problem 10
2.1. Problem statement 10
2.2. Conceptual solution 10
2.3. Problems encountered and how they were fixed 11
2.4. Interesting observations 12
2.5. Results of execution 13
2.6. Test cases 14
2.7. Conclusion 14
3. Hamming Distance and Gray Code 15
3.1. Problem statement 15
3.2. Conceptual solution 15
3.3. Problems encountered and how they were fixed 17
3.4. Interesting observations 17
3.5. Results of execution 18
3.6. Test cases 18
3.7. Conclusion 19
4. Shortest Paths from One Vertex to All Vertices 20
4.1. Problem statement 20
4.2. Conceptual solution 20
4.3. Problems encountered and how they were fixed 21
4.4. Interesting observations 22
4.5. Results of execution 23
4.6. Test cases 24
4.7. Conclusion 24
General Conclusions 26

Introduction

This report describes four laboratory works devoted to genetic algorithms and to the discrete structures they operate on. The first work maximizes a polynomial of four integer variables, the second searches for a shortest Hamiltonian cycle, the third studies Hamming distance and Gray code, and the fourth finds shortest paths from one vertex of a weighted graph to all the others.

One principle runs through all four works: a heuristic result is never accepted on faith. A genetic algorithm returns the best individual it has seen, and by itself that says nothing about whether the true optimum was reached. Therefore every genetic search here is paired with an exact method on the same instance: a full scan of the domain in the first work, enumeration of all tours in the second, and a relaxation algorithm together with enumeration of all simple paths in the fourth. When the two disagree, the disagreement is investigated rather than hidden.

All programs are written in Python using only the standard library, so every operator stays visible and can be followed by hand. Correctness is confirmed by an automated suite of 55 checks covering normal cases, worked examples and invalid inputs. In the final run all 55 checks passed. Beyond the required tasks, each chapter contains small additional experiments that were run to understand why the algorithms behave as they do, for example how the population size or the choice of mutation changes the success rate.

Table 0.1. Overview of the laboratory works

Work	Problem	Size of the search space	Exact reference	Final result
1	Maximize f(w, x, y, z) on integers 0..15	65 536 chromosomes	Full scan of the domain	Maximum 3602 at (15, 0, 15, 15)
2	Travelling salesman, 5 cities	24 chromosomes, 12 distinct cycles	Enumeration of all tours	Shortest tour 0-1-3-2-4, length 26
3	Hamming distance and Gray code	All bit strings up to 8 bits	Two independent constructions	Round trip exact for every string
4	Shortest paths from vertex 1, 7 vertices	All simple paths of the network	Relaxation and path enumeration	Distances 20, 20, 25, 31, 38, 39

1. Maximizing a Function of Four Variables

1.1. Problem statement

The goal of the first work is to find the maximum of the function

$$f(w, x, y, z) = w^3 + x^2 - y^2 - z^2 + 2yz - 3wx + wz - xy + 2$$

where every variable is an integer between 0 and 15. The search has to be performed by a genetic algorithm with a population of 41 individuals, fitness-proportionate selection, crossover and mutation. Because each variable has only sixteen possible values, the whole domain contains $16^4 = 65\,536$ points. This small size is a gift: it lets us compute the true answer independently and judge the genetic algorithm honestly.

1.2. Conceptual solution

Representation. A candidate solution is a chromosome of 16 bits split into four genes of four bits each. Each gene is an unsigned binary number and directly gives one variable. Four bits encode exactly the values 0..15, so every possible bit string is a legal individual and no repair step is ever needed. Figure 1.1 shows how one chromosome is read.



Figure 1.1. The chromosome 1101 0110 0111 1100 decodes to w = 13, x = 6, y = 7, z = 12

Fitness. The fitness of a chromosome is simply the value of f at the decoded point; a larger value is better.

Selection. Parents are chosen by a roulette wheel, where each individual receives a slice proportional to its weight. How those weights are obtained from f is the most delicate part of this work and is discussed in section 1.3.

Crossover. One-point crossover chooses a cut position and exchanges the tails of two parents. Two-point and multi-point variants were also implemented and verified, although the main loop uses the one-point version.

Mutation. Each bit is flipped independently with probability 1/16, so on average exactly one bit of a 16-bit chromosome changes per generation.

Replacement with elitism. Each generation builds a completely new population of 41 individuals. The best chromosome of the current generation is copied unchanged into the new one, and the remaining 40 places are filled with mutated children. The run lasts 200 generations.

A second method for comparison. In addition to the genetic algorithm, a steepest-ascent hill climber was implemented. It starts from a random chromosome, tries flipping each of the 16 bits, moves to the best improving neighbour and restarts from a new random point when no neighbour is better. Finally, a brute-force scan of all 65 536 chromosomes provides the exact maximum against which both heuristics are measured.

1.3. Problems encountered and how they were fixed

Negative fitness breaks the roulette wheel. The function f is not positive: its smallest value on the domain is −341, and, as Figure 1.2 shows, almost a quarter of all chromosomes have negative fitness. A roulette wheel with a negative slice is not a probability distribution, and a naive implementation would either crash or silently favour the worst individuals. The fix was to shift all fitness values of the current population so that the worst individual receives weight exactly 1. For example, for a population with fitness 671, −43, 239 and −91 the weights become 763, 49, 331 and 1. The differences between individuals are preserved, the order is preserved, and even the worst individual keeps a small chance to be selected, which helps diversity.

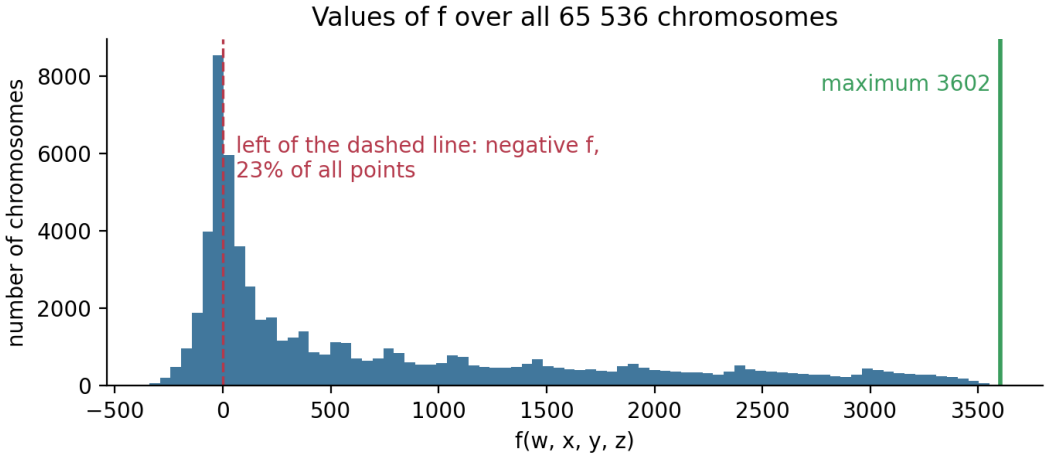


Figure 1.2. Distribution of f over the entire domain

Losing the best solution. If the whole population is replaced by children, a very good chromosome can be found in one generation and destroyed in the next by crossover or mutation, so the best fitness goes down. Copying the best individual

unchanged into the next population (elitism) fixed this. The best fitness now provably never decreases, and the tests check this property on every generation.

Inconsistent values in the reference example. The hand-worked example provided with the task lists fitness values for a few sample chromosomes. When the program was compared with it, two values did not match. For the point (1, 6, 8, 0) the example gives 91, while the formula gives −91; the negative value is the correct one, because the average of the four values in the same example equals 194 only when −91 is used. For the point (11, 6, 14, 11) the example gives 1045, while the formula gives 1199; the average stated for that generation, 412, was clearly computed from the wrong number. The program follows the formula, and the discrepancy is documented rather than copied.

1.4. Interesting observations

The landscape has only one hilltop. An exhaustive check showed that among all 65 536 chromosomes there is exactly one point from which no single-bit flip improves the fitness, and it is the global maximum itself. In other words, under one-bit moves this function has no false local maxima. This explains why the hill climber succeeded in every one of 30 restarts, even with a budget of only 1000 evaluations, and it warns us that this problem is easier than it looks.

Near-ties at the top. The second-best chromosome, (15, 0, 14, 15), scores 3601, only one unit below the optimum, and the third, (15, 0, 13, 15), scores 3598. A run that stopped one bit away from the answer would look almost perfect on a chart, which is a good reason to verify the final value exactly rather than visually.

Mean and best behave differently. Figure 1.3 shows a typical run. The best individual reaches the maximum by generation 16, while the population mean rises quickly and then hovers around 2600–2900 without ever approaching 3602. Mutation keeps injecting weaker individuals, which is exactly what maintains diversity.

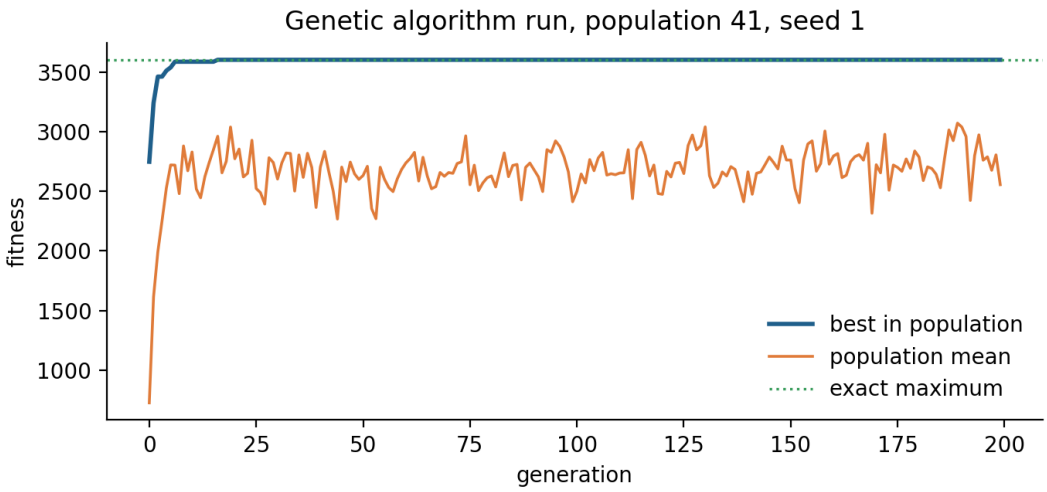


Figure 1.3. Best and mean fitness in one run of the genetic algorithm

Mutation is essential. To see how much mutation matters, the algorithm was run 20 times with three different mutation probabilities (Figure 1.4). Without mutation only 4 runs out of 20 found the maximum: once the population lost a particular bit value, crossover alone could never bring it back. With the chosen probability of 1/16 per bit every run succeeded, while a much higher rate of 0.2 started to behave like random search and missed the optimum once, ending at 3601.

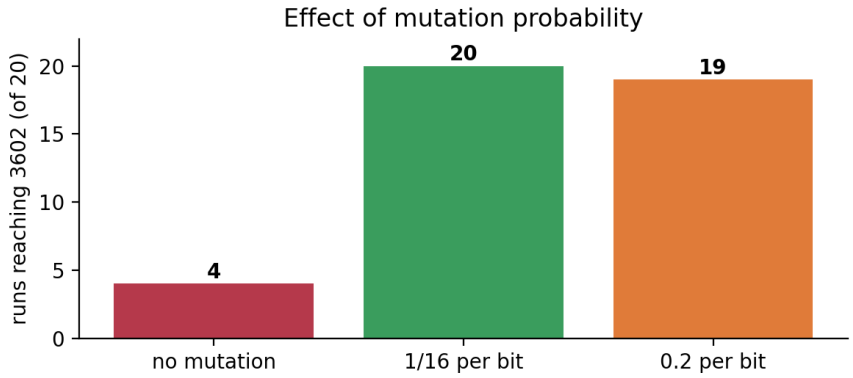


Figure 1.4. Number of successful runs for different mutation probabilities

Does Gray coding help here? Since the third work introduces Gray code, the genes were also decoded through Gray code as an experiment. Over 100 runs both encodings always found the maximum, with a median of 29 generations for plain binary and 32 for Gray. Because this landscape has no deceptive local maxima, the usual advantage of Gray code simply has nothing to fix.

1.5. Results of execution

Table 1.1. Results of the three methods

Method	Settings	Chromosome found	Point (w, x, y, z)	Fitness
Exhaustive scan	all 65 536 points	1111 0000 1111 1111	(15, 0, 15, 15)	3602
Genetic algorithm	population 41, 200 generations	1111 0000 1111 1111	(15, 0, 15, 15)	3602
Steepest ascent	budget of 8000 evaluations	1111 0000 1111 1111	(15, 0, 15, 15)	3602

To make sure the result was not a lucky accident, the genetic algorithm was run with 30 different random seeds. All 30 runs ended at 3602. The generation in which the maximum first appeared ranged from 8 to 82, with a median of about 30. So the 200 generations used by default leave a comfortable safety margin.

1.6. Test cases

Table 1.2. Selected verification cases

Case	Expected	Observed
f(0, 0, 0, 0)	2	passed
f(10, 14, 8, 3) and f(6, 9, 15, 6)	671 and −43	passed
f(1, 6, 8, 0), disputed value	−91	passed
f(11, 6, 14, 11), disputed value	1199	passed
Average of 671, −43, 239, −91	194	passed
Decoding 1101011001111100	(13, 6, 7, 12)	passed
Chromosome of 15 or 17 bits, or containing a non-binary symbol	rejected	passed
Variable equal to −1 or 16	rejected	passed
One-point crossover of 11110000 and 00001111 at position 4	11111111 and 00000000	passed
Roulette weights when some fitness values are negative	all weights positive	passed
Best fitness over 200 generations	never decreases	passed
Genetic algorithm and hill climber versus exhaustive scan	both reach 3602	passed

1.7. Conclusion

The genetic algorithm reliably finds the global maximum f(15, 0, 15, 15) = 3602, confirmed by a complete scan of the domain. The work showed that a roulette wheel needs non-negative weights and that a simple shift solves this cleanly, that elitism is necessary to keep the best result, and that mutation is what keeps the search alive. It also showed that this particular landscape is benign: it has a single hilltop, so even plain hill climbing succeeds. That is a useful reminder that the success of a genetic algorithm must always be judged against the difficulty of the problem.

2. The Travelling Salesman Problem

2.1. Problem statement

Given a complete undirected graph with $n \geq 3$ cities and a symmetric matrix of non-negative distances, find a Hamiltonian cycle of minimum length: a tour that visits every city exactly once and returns to the starting city. The length of a tour includes the final edge back to the start. The working instance contains five cities with the distances shown in Table 2.1.

Table 2.1. Distance matrix of the five-city instance

	City 0	**City 1**	**City 2**	**City 3**	**City 4**
City 0	0	2	9	10	7
City 1	2	0	6	4	3
City 2	9	6	0	8	5
City 3	10	4	8	0	6
City 4	7	3	5	6	0

2.2. Conceptual solution

Representation. A chromosome is a permutation of the cities, that is, the order in which they are visited. Since a cycle can start anywhere, every tour is stored rotated so that it begins with city 0. With the start fixed there are $(n - 1)!$ possible chromosomes; because a cycle can also be travelled in either direction, the number of genuinely different tours is $(n - 1)!/2$. For five cities these numbers are 24 and 12.

Fitness. The tour length; a smaller length is better.

Selection. In each generation the two shortest tours of the population are chosen as parents.

Crossover. Ordinary crossover cannot be used on permutations, as section 2.3 shows. Instead, order crossover is used: two cut points are chosen, and inside the cut segment each parent keeps its own set of cities but rearranges them in the order in which they appear in the other parent. Figure 2.1 shows the operation on two eight-city parents. The first parent's segment contains cities 6, 1, 0, 2; in the second parent these cities appear in the order 1, 2, 6, 0, and that becomes the new segment of the first child.

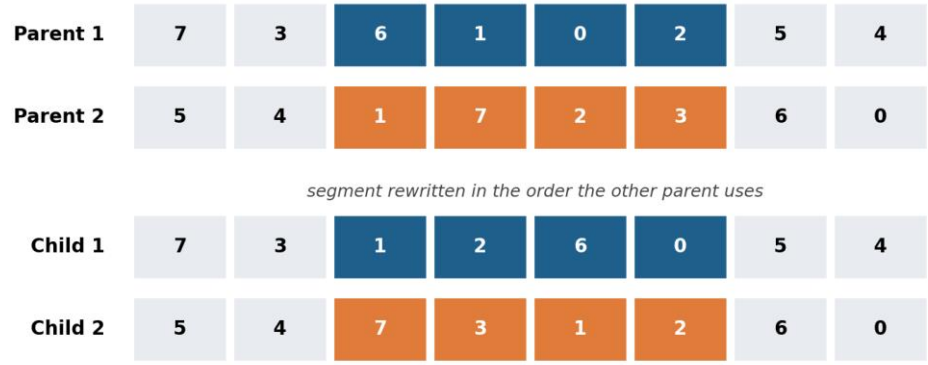


Figure 2.1. Order crossover: each child keeps its own cities in the segment, reordered as in the other parent

Mutation. With probability 0.1 a child undergoes an adjacent swap: a randomly chosen city exchanges places with the next one. The last city is considered adjacent to the first, so the result is always a valid tour. Two alternative operators, swapping two arbitrary cities and reversing a segment, were also implemented; the reversal later turned out to be important (section 2.4).

Replacement. The two children overwrite the two longest tours of the population. The parents stay, so good tours are never lost. The algorithm also remembers the shortest tour ever seen. The population size is 30 and the run lasts 300 generations.

2.3. Problems encountered and how they were fixed

Plain crossover produces invalid tours. The obvious approach is to swap the cut segments of the two parents directly, as in the bit-string case. For the parents of Figure 2.1 this produces 7 3 1 7 2 3 5 4, in which cities 7 and 3 appear twice while cities 6 and 0 disappear. Such a string is not a tour at all. Order crossover solves this by construction: each child keeps exactly the cities it already had, only their order changes, so every child is a valid permutation.

Losing the best tour. One natural reading of the replacement step is that the children replace their parents. But the parents are always the two best tours, so that rule would delete the current best solution in every generation. Replacing the two worst tours instead keeps the best tours in the population and makes the search monotone.

Rotation of tours. Order crossover can move city 0 away from the first position. Since rotating a cycle does not change its length, every child is simply rotated back so that it starts at city 0, which keeps the representation canonical.

Input validation. The program rejects matrices with fewer than three cities, negative distances, a non-zero diagonal or asymmetric entries, and tours that repeat or

skip a city. One reference figure supplied with the task stated tour lengths of 33 without giving its distance matrix; rather than inventing a matrix that fits the answer, that case is reported as missing data.

2.4. Interesting observations

The five-city instance is too small for a genetic algorithm. There are only 24 chromosomes in total, and the population holds 30 random tours. As a result the initial population almost certainly contains an optimal tour already, and in all 30 test runs the best length was 26 from the very first generation. Figure 2.2 shows the distribution of all 24 tour lengths: four chromosomes (two cycles, each in two directions) reach the optimum of 26.

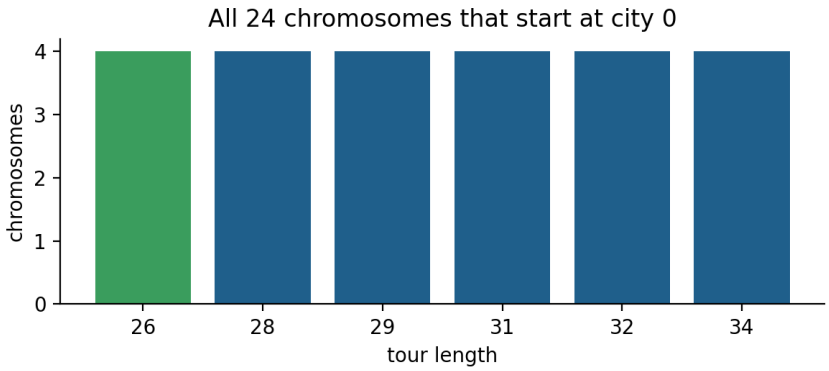


Figure 2.2. Lengths of all 24 chromosomes of the five-city instance

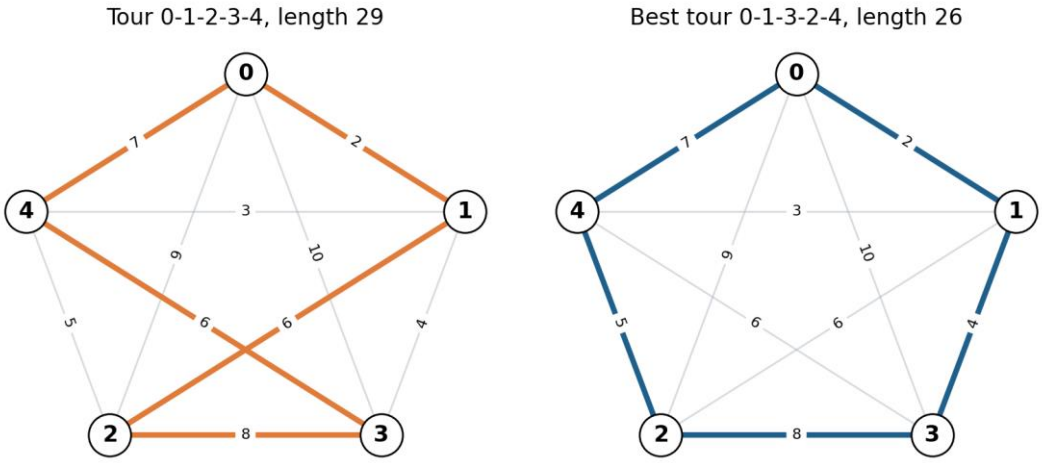


Figure 2.3. A non-optimal tour of length 29 compared with the optimal tour of length 26

A harder test with nine cities. To see the algorithm actually working, an additional instance of nine randomly placed cities was generated. It has 40 320 chromosomes, and exhaustive search shows that the optimal tour has length 254. Here the picture changed sharply: with the adjacent swap only 5 of 30 runs found the

optimum, and the average result was 292.7. Running for 1000 generations instead of 300 did not help at all, because the population had already converged and only the two worst tours change per generation.

The choice of mutation matters more than the number of generations. An adjacent swap is a very small move; it cannot untangle two crossing edges of a tour. Reversing a segment can, because it is exactly the move that removes a crossing. With segment reversal at probability 0.5, 22 of 30 runs found the optimal length 254 and the worst result was 260. Figure 2.4 compares the two variants.

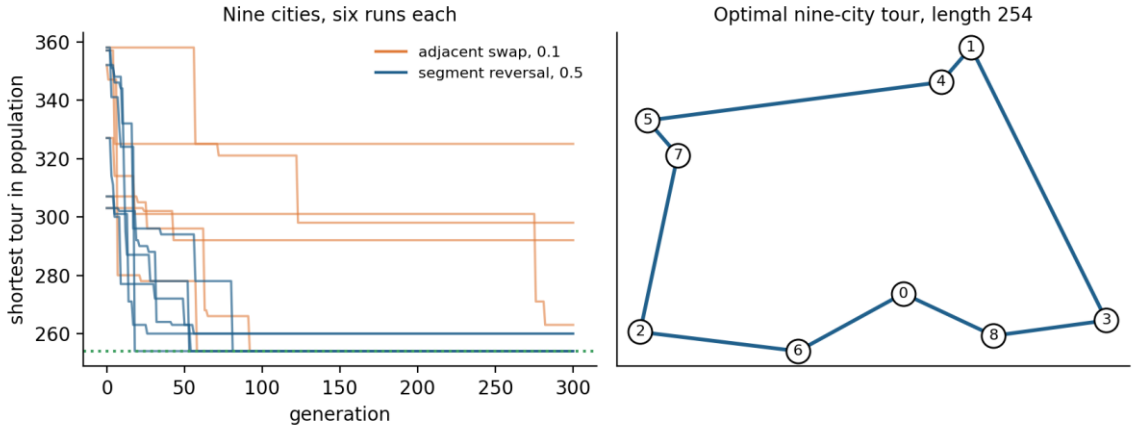


Figure 2.4. Nine-city experiment: progress of six runs per mutation operator (left) and the optimal tour (right)

Table 2.2. Nine-city instance, 30 runs per configuration (optimum 254)

Mutation	Probability	Generations	Runs reaching 254	Average length	Worst length
Adjacent swap	0.1	300	5	292.7	356
Adjacent swap	0.5	300	4	294.5	356
Adjacent swap	0.1	1000	5	292.6	356
Segment reversal	0.1	300	13	266.1	306
Segment reversal	0.5	300	22	255.6	260

2.5. Results of execution

On the five-city instance the genetic algorithm returns the tour 0 → 1 → 3 → 2 → 4 → 0 with length 2 + 4 + 8 + 5 + 7 = 26. Exhaustive enumeration of all 24 chromosomes confirms that 26 is the minimum. The reverse cycle 0 → 4 → 2 → 3 →

1 → 0 has the same length, since it is the same tour travelled the other way. All 30 runs with different seeds returned length 26.

2.6. Test cases

Table 2.3. Selected verification cases

Case	Expected	Observed
Length of tour 0-1-2-3-4, including the return edge	2 + 6 + 8 + 6 + 7 = 29	passed
Order crossover on the parents of Figure 2.1	children 7 3 1 2 6 0 5 4 and 5 4 7 3 1 2 6 0	passed
Plain segment swap on the same parents	repeats cities, rejected as a tour	passed
Adjacent swap of city 6 in 7 3 1 2 6 0 5 4	7 3 1 2 0 6 5 4	passed
Number of chromosomes and of distinct cycles for n = 5	24 and 12	passed
Selection of parents	the two shortest tours	passed
Fewer than 3 cities, negative or asymmetric distances	rejected	passed
Tour that visits a city twice	rejected	passed
Genetic algorithm versus enumeration	both give 26	passed

2.7. Conclusion

The genetic algorithm with order crossover finds the optimal five-city tour of length 26, confirmed by complete enumeration. The main conceptual lesson was that permutation problems need permutation-preserving operators: naive crossover destroys the solution structure, while order crossover keeps it intact. The additional nine-city experiment revealed the real weakness of the basic design: with only the two best tours as parents and a tiny mutation the population converges early, and more generations do not help. A stronger mutation, segment reversal, raised the success rate from 17% to 73%.

3. Hamming Distance and Gray Code

3.1. Problem statement

The third work has two parts. The first is to compute the Hamming distance between two strings of equal length and to verify that it is a metric. The second is to convert bit strings from ordinary binary to binary reflected Gray code and back. The main acceptance criterion is the round trip: converting a binary string to Gray code and then back must return exactly the original string. In addition, neighbouring codes in the Gray sequence, including the last and the first, must differ in exactly one bit.

Although this work contains no genetic search, it is closely related to the others: Hamming distance measures how far apart two chromosomes are, and Gray code is a classic alternative way of encoding integer genes such as those in the first work.

3.2. Conceptual solution

Hamming distance is the number of positions in which two strings differ. The symbols can be letters, digits or bits. Strings of different length have no Hamming distance, so such input is rejected. The three metric axioms are: the distance of a string to itself is zero; the distance is symmetric; and the triangle inequality $d(a, b) \le d(a, c) + d(c, b)$ holds. The proof of the triangle inequality is checked position by position: if a and b differ at some position, then c must differ from at least one of them there, so that position contributes at least as much to the right side as to the left. Summing over all positions gives the inequality.

Table 3.1. Hamming distance examples

First string	Second string	Distance
karolin	kathrin	3
karolin	kerstin	3
kathrin	kerstin	4
0000	1111	4
2173896	2233796	3

Gray code by reflection. The one-bit Gray code is simply 0, 1. To build the code of length n + 1, take the code of length n with a 0 in front, and then the same code written in reverse order with a 1 in front. Figure 3.1 shows how the three-bit code is obtained from the two-bit one. The two halves are mirror images, so at the junction the

strings differ only in the first bit, and the same is true at the wrap-around from the last string back to the first.

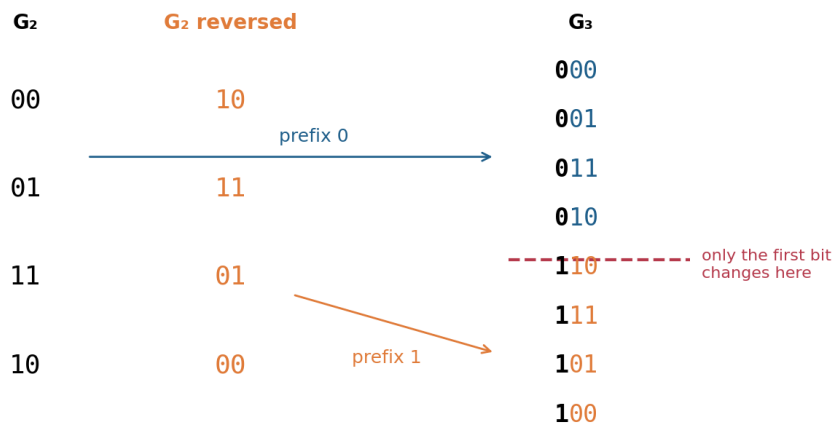


Figure 3.1. Construction of the three-bit Gray code by reflection

Gray code by a bit rule. The same code can be computed directly: the first Gray bit equals the first binary bit, and every following Gray bit is the exclusive OR of the current and the previous binary bit. The inverse conversion accumulates the exclusive OR from left to right: each binary bit is the exclusive OR of the previous binary bit and the current Gray bit.

Why two constructions? A single implementation would be shorter, but a subtle error in it could still produce a list in which neighbours differ by one bit, so that error would go unnoticed. Having two independent constructions and requiring that they agree on every string of length 1 to 8 makes such an error visible immediately.

Table 3.2. Three-bit binary and Gray codes with the round trip

Decimal	Binary	Gray code	Back to binary	Bits changed from previous Gray code
0	000	000	000	1 (from 100)
1	001	001	001	1
2	010	011	010	1
3	011	010	011	1
4	100	110	100	1
5	101	111	101	1
6	110	101	110	1
7	111	100	111	1

3.3. Problems encountered and how they were fixed

Two orders of bits. The bit rule can be written with the most significant bit on the left or on the right, and mixing the two conventions gives a code that still looks plausible but does not match the reflected list. Requiring both constructions to agree for all strings up to 8 bits fixes the convention unambiguously: the leftmost character is the most significant bit.

Invalid input. Empty strings and strings containing symbols other than 0 and 1 are rejected in both directions, and so are attempts to build a code of length zero or to compare strings of different length. Without these checks a stray character such as "2" would be silently treated as a 1.

3.4. Interesting observations

The switch example. Going from 3 to 4 in ordinary binary means changing 011 into 100, which flips all three bits at once. If the bits were physical switches, they would never flip at exactly the same moment, and the device could briefly read any intermediate value. In Gray code the same step is 010 → 110, a single switch. This is the original practical motivation for Gray code.

Exactly half the switching. Counting through all 2^n values and returning to zero, Gray code flips one bit per step, 2^n flips in total. Ordinary binary flips $2^{n+1} - 2$ bits, almost twice as many, and its worst single step flips all n bits. Figure 3.2 compares the two on a logarithmic scale.

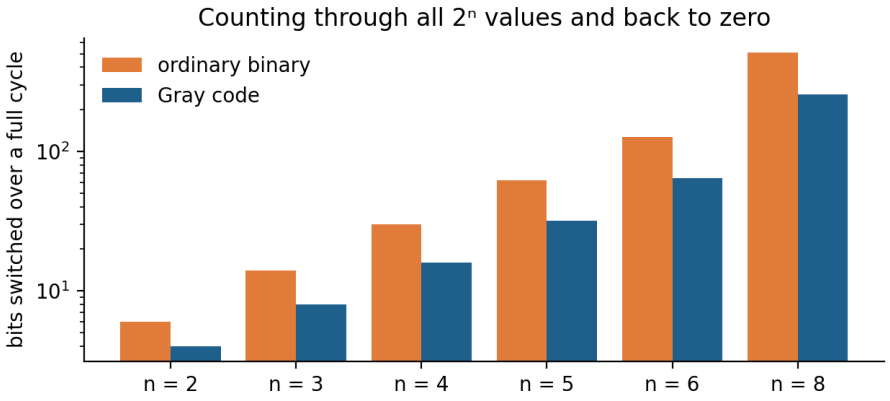


Figure 3.2. Total number of bit changes when counting through a full cycle

Gray code as a walk on the hypercube. The bit strings of length n are the vertices of an n-dimensional cube, where two strings are joined by an edge exactly when their Hamming distance is 1. The Gray code is a cycle that visits every vertex of this cube once, a Hamiltonian cycle. For n = 3 the cube has 12 edges and the Gray cycle uses 8 of them (Figure 3.3). In general the cube has $n \cdot 2^{n-1}$ edges, so the remaining

edges connect strings that are adjacent in Hamming distance but not consecutive in the code.

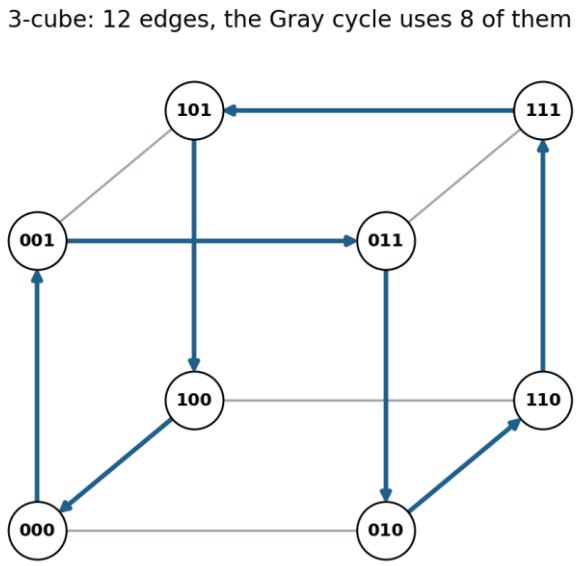


Figure 3.3. The three-dimensional cube and the Gray cycle on it

Link to the genetic algorithm. In the first work, the step from gene value 7 (0111) to 8 (1000) requires four simultaneous mutations in binary but only one in Gray code. Such "Hamming cliffs" are the classic argument for Gray-coded genes. The experiment described in section 1.4 showed, however, that for that particular function the effect is negligible, because its landscape has no local traps.

3.5. Results of execution

The reflected construction produced 0, 1 for one bit; 00, 01, 11, 10 for two bits; and 000, 001, 011, 010, 110, 111, 101, 100 for three bits. For every length from 1 to 8 and every value, the bit rule gave the same code as the reflected list, and converting back returned the original binary string; in total 510 strings were checked. In every list, every pair of neighbours and the pair formed by the last and the first string differ in exactly one bit. Each list is a permutation of all values, and the first half of the code of length n + 1 is the code of length n with a leading zero.

3.6. Test cases

Table 3.3. Selected verification cases

Case	Expected	Observed
Five Hamming distance examples of Table 3.1	3, 3, 4, 4, 3	passed

Case	Expected	Observed
Distance of a string to itself, symmetry	0, equal in both orders	passed
Triangle inequality for all triples of the three words	holds position by position	passed
Strings of different length	rejected	passed
Gray codes of length 1, 2 and 3	the reflected lists	passed
Round trip for every string of length 1 to 8	original string returned	passed
Neighbours and the cyclic pair	differ in one bit	passed
Edges of the three-dimensional cube	12 edges, all at distance 1	passed
Empty string, non-binary symbol, length zero	rejected	passed

3.7. Conclusion

Hamming distance was implemented and verified as a metric, and binary reflected Gray code was implemented in two independent ways that agree on every string up to eight bits. The required round trip from binary to Gray and back is exact. The work also connected the code to geometry, as a Hamiltonian cycle on the hypercube, and to genetic algorithms, where it removes Hamming cliffs between neighbouring integer values.

4. Shortest Paths from One Vertex to All Vertices

4.1. Problem statement

Given a weighted graph and a source vertex, find the shortest path from the source to every other vertex. The work requires two approaches: an exact one, based on iterative relaxation of distances, and a genetic algorithm whose individuals are paths. The instance is an undirected network of seven vertices taken from a published study of genetic algorithms for routing (Figure 4.1). The source is vertex 1.

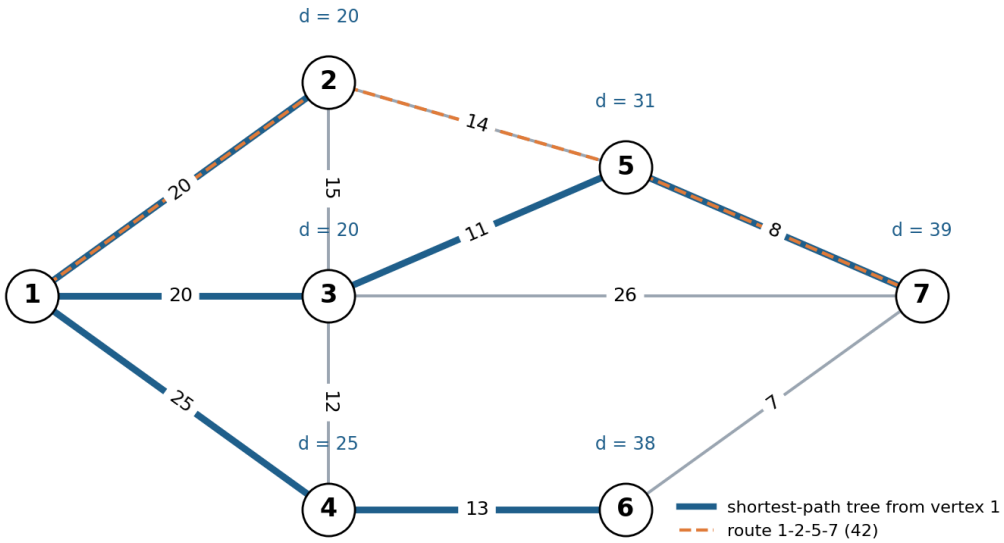


Figure 4.1. The seven-vertex network, the shortest-path tree from vertex 1 and the route 1-2-5-7

4.2. Conceptual solution

Exact method: synchronous relaxation. Initially the distance of the source is 0 and every other distance is infinite. In each round, every vertex looks at all its incoming edges and takes the smaller of its current distance and the distance of a neighbour plus the edge weight. All updates in a round use the values from the previous round only. After as many rounds as there are vertices, every distance is exact, provided the weights are non-negative. Table 4.1 shows how the distances evolve; after the third round nothing changes any more.

Table 4.1. Distances from vertex 1 after each round of relaxation

Round	v1	v2	v3	v4	v5	v6	v7
0	0	∞	∞	∞	∞	∞	∞
1	0	20	20	25	∞	∞	∞
2	0	20	20	25	31	38	46

Round	v1	v2	v3	v4	v5	v6	v7
3 to 7	0	20	20	25	31	38	39

Notice vertex 7 in round 2: it first receives distance 46 via the direct edge 3 → 7, and only in round 3, once vertex 5 is known, is this improved to 39 via 5 → 7. This is exactly why a single round is not enough.

Genetic algorithm. An individual is a simple path from the source to one target: it starts at the source, ends at the target, never visits a vertex twice and uses only existing edges. Its cost is the sum of the edge weights. The initial population of 15 paths is built by random walks that step back whenever they reach a dead end.

Crossover chooses a vertex that lies on both parents and exchanges the parts of the paths after it (Figure 4.2). Mutation inserts a random vertex into a random interior position of a path. If either operator produces an invalid path, the child is discarded and the parent is kept.

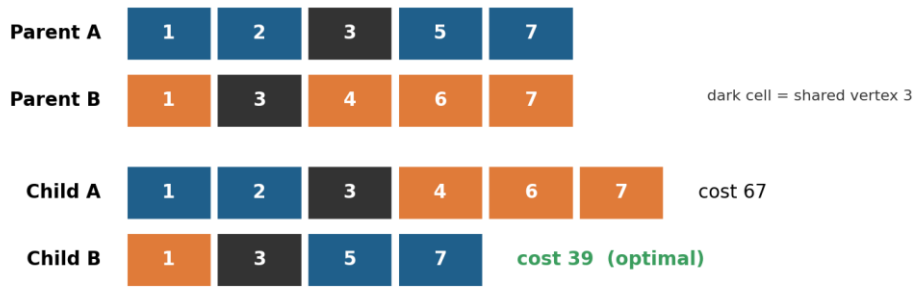


Figure 4.2. Crossover of two paths at their common vertex 3

Selection uses a roulette wheel, the best path is carried over to the next generation, and the search stops when the best cost has not improved for 10 generations or after 100 generations, whichever comes first. To obtain paths to all vertices, the genetic algorithm is run once for each of the six targets.

4.3. Problems encountered and how they were fixed

A missing edge disguised as a long edge. In the original data a missing edge is written as the number 1000. If this number is treated as a weight, a path through a non-existent edge simply looks expensive rather than impossible, and on a sparse graph it could even be returned as the answer. The program stores a missing edge as truly absent and rejects any path that tries to use it.

Selection pointing the wrong way. The published method defines the selection probability as proportional to the path cost. But a lower cost is better, so this formula would choose a path of cost 42 more often than a path of cost 39. The weight was changed to the reciprocal of the cost, so shorter paths receive larger slices of the wheel.

Ambiguous stopping rule. The published pseudocode continues while the generation limit or the stagnation limit has not been reached, which means it stops only when both are reached. The accompanying text says the search ends when either condition is met. The program follows the text: for example, with a limit of 100 generations and 10 generations without improvement already reached at generation 10, it stops immediately instead of running 90 more generations for nothing.

One chromosome cannot hold all answers. A path chromosome connects the source to a single target, while the task asks for paths to every vertex. Rather than inventing a new encoding for a whole shortest-path tree, the genetic algorithm is run separately for each target, and each answer is compared with the relaxation result.

4.4. Interesting observations

The "optimal" route of the published example is not optimal. The source study lists eleven routes from vertex 1 to vertex 7 and presents 1 → 2 → 5 → 7 with cost 42 as the best. All eleven costs were recomputed from the matrix and they are correct, but the route 1 → 3 → 5 → 7 with cost 20 + 11 + 8 = 39 is cheaper and does not appear in the list at all. Both the relaxation and an independent enumeration of all simple paths confirm 39 as the true minimum.

Table 4.2. Routes from vertex 1 to vertex 7, recomputed from the matrix

Route	Cost	Route	Cost
1-3-5-7 (not in the published list)	39	1-2-3-5-7	54
1-2-5-7 (published as best)	42	1-4-3-5-7	56
1-4-6-7	45	1-2-3-7	61
1-3-7	46	1-4-3-7	63
1-3-4-6-7	52	1-2-3-4-6-7	67
1-2-5-3-7	71	1-2-5-3-4-6-7	77

The genetic algorithm can fall into the same trap. Over 50 runs with different seeds, 48 found the cost 39. The two failing runs both stopped at exactly 1 → 2 → 5

→ 7 with cost 42, the same route the published study called optimal. In both runs the best initial path was 1 → 2 → 5 → 7, and most other paths shared with it only the two endpoints, so crossover had no common interior vertex to cut at and could only reproduce the parents. The optimal route could still appear through mutation, for example by inserting vertex 5 into 1 → 3 → 7, but that did not happen within ten generations and the stagnation rule ended the search one step before success. A further reason is weak selection pressure: with reciprocal weights a path of cost 39 is only about 8% more likely to be chosen than one of cost 42.

The population size matters a lot. Figure 4.3 shows the share of 50 runs finding the optimum for different population sizes. With only 2 paths the algorithm succeeded in 18% of runs, with 5 paths in 50%, and with the prescribed 15 paths in 96%.

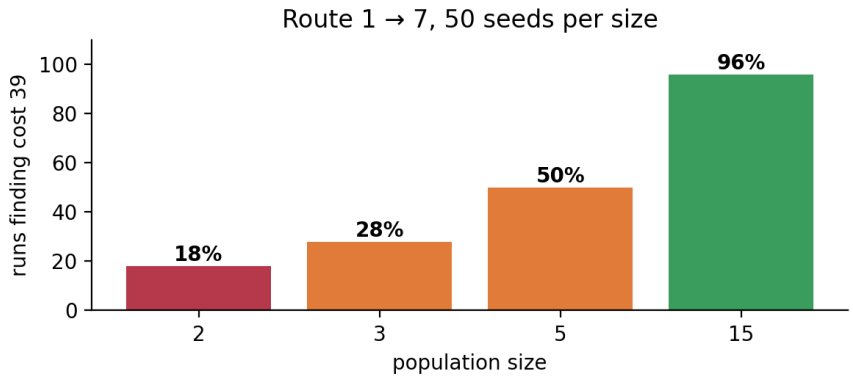


Figure 4.3. Success rate of the genetic algorithm for the route 1 → 7 depending on population size

The network is small. With 15 random walks per target, the initial population usually already contains the shortest path, so most runs stop at generation 10 without any improvement. On a larger graph the generation limit, the stagnation limit and the rejection of invalid children would all need to be tuned; on a sparse graph many crossovers and insertions would be rejected, and the population could freeze while the stagnation counter keeps growing.

4.5. Results of execution

Table 4.3. Shortest paths from vertex 1

Target	Path found by the genetic algorithm	Cost	Relaxation distance	Generations run
2	1 → 2	20	20	10
3	1 → 3	20	20	10
4	1 → 4	25	25	10

Target	Path found by the genetic algorithm	Cost	Relaxation distance	Generations run
5	1 → 3 → 5	31	31	10
6	1 → 4 → 6	38	38	10
7	1 → 3 → 5 → 7	39	39	10

For every target the path returned by the genetic algorithm has exactly the cost computed by relaxation, and both agree with exhaustive enumeration of simple paths. Each shortest path in this network is unique. Together the six paths form the shortest-path tree highlighted in Figure 4.1.

4.6. Test cases

Table 4.4. Selected verification cases

Case	Expected	Observed
Costs of all eleven published routes	match the matrix, including 42	passed
Distances after round 0 and round 1	only the source is 0; then 20, 20, 25 for vertices 2-4	passed
Relaxation versus enumeration of all simple paths	identical distances	passed
Small directed graph with edges 0→1 (1), 0→2 (4), 1→2 (1)	distances 0, 1, 2	passed
Crossover of 1-2-3-5-7 and 1-3-4-6-7 at vertex 3	1-2-3-4-6-7 and 1-3-5-7	passed
Insertion of vertex 5 into 1-2-3-7	1-2-3-5-7 with cost 54	passed
Path through a missing edge, repeated vertex, wrong end	rejected	passed
Roulette weights for costs 39, 42, 46	strictly decreasing	passed
Stopping after 10 stagnant generations or the limit	stops on either	passed
Genetic algorithm for every target	equals relaxation distance	passed

4.7. Conclusion

Both methods find the same shortest paths from vertex 1: 20, 20, 25, 31, 38 and 39 to vertices 2 through 7. Relaxation does so exactly in three effective rounds; the genetic algorithm does so by a population search over simple paths, with a 96% success

rate for the hardest target. The most valuable outcome of this work is critical rather than computational: by checking the published example against an exact method, an error in it was found, and the genetic algorithm was observed to stall on exactly that same suboptimal route. Fixing the treatment of missing edges, the direction of selection and the stopping rule were all necessary for a correct implementation.

General Conclusions

Four laboratory works were completed, and in every one of them the result of the heuristic search was confirmed by an exact method. The genetic algorithm found the maximum 3602 of the four-variable function, the shortest five-city tour of length 26, and all six shortest paths in the seven-vertex network. Gray code was implemented in two independent ways whose round trip is exact for every string up to eight bits.

Several lessons repeat across the works. First, the operators must respect the structure of the solution: a roulette wheel needs non-negative weights, a tour needs permutation-preserving crossover, and a path needs crossover at a shared vertex with rejection of invalid children. Second, elitism or an equivalent mechanism is needed so that the best solution is never lost. Third, diversity is decisive: removing mutation, using too weak a mutation, or using too small a population consistently caused the search to stall, while more generations did not help.

Finally, the works showed the value of independent checking. Comparing results with exhaustive search revealed inconsistent numbers in a reference example, a suboptimal answer presented as optimal in a published study, and the fact that the first problem, despite its size, has only a single hilltop. Without the exact references none of these would have been noticed, and a genetic algorithm reporting a good-looking number would have been taken at its word.

All 55 automated checks passed in the final run.